

1. Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

Answer:

Query processing converts an SQL query into an executable plan and produces the required result.

Pseudocode:

ALGORITHM Query_Processing(SQL_Query)

BEGIN

 // Step 1: Receive SQL Query

 Input SQL_Query

 // Step 2: Parsing

 Tokens $\leftarrow$ Lexical_Analyze(SQL_Query)

 ParseTree $\leftarrow$ Syntax_Analyze(Tokens)

 IF ParseTree is INVALID THEN

 Display "Syntax Error"

 STOP

 END IF

 // Step 3: Semantic Analysis

 Check table names

 Check column names

 Check data types

 Check user permissions

 IF Semantic_Error THEN

 Display "Semantic Error"

 STOP

 END IF

 // Step 4: Query Translation

 Relational_Algebra $\leftarrow$ Convert_To_Relational_Algebra(ParseTree)

 // Step 5: Query Optimization

 Candidate_Plans $\leftarrow$ Generate_Execution_Plans(Relational_Algebra)

Best_Plan $\leftarrow$ Plan with minimum estimated cost

// Step 6: Query Execution

Result $\leftarrow$ Execute(Best_Plan)

// Step 7: Output

Display Result

END

Pipeline:

SQL Query



Lexical Analysis



Syntax Parsing



Semantic Analysis



Relational Algebra



Query Optimization



Best Execution Plan



Query Execution



Result

Conclusion: Query processing improves database performance by converting SQL into an efficient execution plan before executing it.

2. Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

Answer:

Cost-based optimization generates different execution plans and selects the plan with the lowest estimated cost.

Pseudocode:

ALGORITHM Cost_Based_Join_Optimization(Tables)

BEGIN

BestPlan $\leftarrow$ NULL

MinimumCost $\leftarrow$ INFINITY

Generate all possible join orderings of Tables

FOR each Ordering in Join_Orderings DO

CurrentPlan $\leftarrow$ NULL

CurrentCost $\leftarrow$ 0

FOR each Join in Ordering DO

Estimate number of tuples in both relations

Estimate number of pages in both relations

// Estimate I/O cost

JoinCost $\leftarrow$ Estimate_Join_IO_Cost(Join)

CurrentCost $\leftarrow$ CurrentCost + JoinCost

Add Join to CurrentPlan

END FOR

IF CurrentCost < MinimumCost THEN

MinimumCost $\leftarrow$ CurrentCost

BestPlan $\leftarrow$ CurrentPlan

END IF

END FOR

RETURN BestPlan, MinimumCost

END

Example:

Suppose there are three tables:

A JOIN B JOIN C

Possible join orders:

(A JOIN B) JOIN C

(A JOIN C) JOIN B

(B JOIN C) JOIN A

The optimizer estimates the I/O cost of each plan and selects the cheapest one.

Conclusion: Cost-based optimization reduces disk I/O by selecting the join ordering with the minimum estimated execution cost.

3. Write a pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

Answer:

Nested Loop Join compares tuples of two relations. One relation is treated as the outer relation and the other as the inner relation.

Pseudocode:

ALGORITHM Nested_Loop_Join(R, S, Join_Condition)

BEGIN

Result $\leftarrow$ Empty

FOR each tuple r in R DO

```

FOR each tuple s in S DO

    IF Join_Condition(r, s) = TRUE THEN
        Add (r, s) to Result
    END IF

END FOR

END FOR

RETURN Result

END

```

Block Nested Loop Join:

If $B(R)$ is the number of pages in R , $B(S)$ is the number of pages in S , and M is the number of available buffer pages:

$$\text{Cost} \approx B(R) + \text{ceil}(B(R)/(M-2)) \times B(S)$$

where:

$B(R)$ = number of pages in relation R

$B(S)$ = number of pages in relation S

M = number of buffer pages

For tuple-based nested loop join:

$$\text{Time Complexity} = O(|R| \times |S|)$$

where:

$|R|$ = number of tuples in R

$|S|$ = number of tuples in S

Conclusion: Nested Loop Join is simple but can be expensive when both relations contain a large number of tuples.

4. Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

Answer:

A Wait-For Graph represents transactions as nodes. An edge $T_i \rightarrow T_j$ means transaction T_i is waiting for a resource held by T_j .

A cycle in the WFG indicates a deadlock.

Pseudocode:

ALGORITHM Detect_Deadlock(WFG)

BEGIN

FOR each transaction T DO

 Visited[T] $\leftarrow$ FALSE

 RecursionStack[T] $\leftarrow$ FALSE

END FOR

FOR each transaction T DO

 IF Visited[T] = FALSE THEN

 IF DFS_Cycle_Detection(T) = TRUE THEN

 Display "Deadlock Detected"

 RETURN TRUE

 END IF

 END IF

END FOR

Display "No Deadlock"

RETURN FALSE

END

ALGORITHM DFS_Cycle_Detection(T)

```

BEGIN

    Visited[T] ← TRUE
    RecursionStack[T] ← TRUE

    FOR each transaction U connected from T DO

        IF Visited[U] = FALSE THEN

            IF DFS_Cycle_Detection(U) = TRUE THEN
                RETURN TRUE
            END IF

            ELSE IF RecursionStack[U] = TRUE THEN
                RETURN TRUE
            END IF

        END FOR

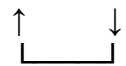
        RecursionStack[T] ← FALSE
        RETURN FALSE

    END

```

Example:

$T1 \rightarrow T2$



Here:

$T1 \rightarrow T2 \rightarrow T1$

forms a cycle, so a deadlock exists.

Conclusion: Cycle detection in a WFG is used to identify deadlocks among transactions.

5. Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

Answer:

Two-Phase Locking (2PL) divides transaction execution into two phases:

1. Growing Phase – locks can be acquired but not released.
2. Shrinking Phase – locks can be released but no new locks can be acquired.

Pseudocode:

ALGORITHM Basic_2PL(Transaction T)

BEGIN

Phase ← GROWING

FOR each operation O of transaction T DO

IF O is READ(X) THEN

Request Shared_Lock(X)

IF Lock is Available THEN

Grant Shared_Lock(X)

Read X

ELSE

Wait until Lock becomes available

Grant Shared_Lock(X)

Read X

END IF

ELSE IF O is WRITE(X) THEN

Request Exclusive_Lock(X)

IF Lock is Available THEN

Grant Exclusive_Lock(X)

Write X

ELSE

Wait until Lock becomes available

Grant Exclusive_Lock(X)


```

        Write X
    END IF

END IF

END FOR

// Begin shrinking phase
Phase ← SHRINKING

FOR each lock held by T DO
    Release Lock
END FOR

COMMIT T

END

```

Two Phases:

Growing Phase



Acquire Locks



Lock Point



Shrinking Phase



Release Locks



Commit

Conclusion: Basic 2PL ensures conflict serializability by preventing a transaction from acquiring new locks after it starts releasing locks.

6. Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

Answer:

Timestamp ordering assigns each transaction a unique timestamp.

For each data item X:

$RTS(X)$ = Largest timestamp of a transaction that read X

$WTS(X)$ = Largest timestamp of a transaction that wrote X

Thomas' Write Rule allows certain obsolete write operations to be ignored instead of aborting the transaction.

Pseudocode:

ALGORITHM Thomas_Write_Rule(T, Operation, X)

BEGIN

TS $\leftarrow$ Timestamp(T)

IF Operation = READ(X) THEN

IF TS < WTS(X) THEN

Abort T

RETURN

ELSE

Perform READ(X)

$RTS(X) \leftarrow \text{MAX}(RTS(X), TS)$

END IF

ELSE IF Operation = WRITE(X) THEN

IF TS < RTS(X) THEN

Abort T

RETURN

ELSE IF TS < WTS(X) THEN

// Thomas Write Rule

Ignore WRITE(X)

RETURN

```

ELSE
    Perform WRITE(X)
    WTS(X) ← TS
END IF

END IF

END

```

Important condition:

If: $TS(T) < WTS(X)$

During a write operation, the write is considered obsolete and can be ignored under Thomas' Write Rule.

Conclusion: Thomas' Write Rule improves concurrency by avoiding unnecessary transaction aborts for obsolete writes.

7. Write a pseudocode algorithm for the ARIES recovery technique with Analysis, Redo, and Undo phases.

Answer:

ARIES stands for Algorithm for Recovery and Isolation Exploiting Semantics. It uses write-ahead logging and performs recovery in three major phases:

Analysis → Redo → Undo

Pseudocode:

ALGORITHM ARIES_Recovery(Log)

BEGIN

 // PHASE 1: ANALYSIS

 Read Log from last checkpoint

 Initialize Transaction Table

 Initialize Dirty Page Table

 FOR each log record DO

```

    IF record represents transaction start THEN
        Add transaction to Transaction Table

    ELSE IF record represents page update THEN
        Add page to Dirty Page Table
        Update Transaction Table

    ELSE IF record represents COMMIT THEN
        Mark transaction as committed

    ELSE IF record represents END THEN
        Remove transaction from Transaction Table

    END IF

END FOR

Identify:
    Winner Transactions
    Loser Transactions

// PHASE 2: REDO
Start from Redo Starting Point

FOR each applicable log record DO

    IF operation needs REDO THEN
        Reapply operation
    END IF

END FOR

// PHASE 3: UNDO
FOR each Loser Transaction DO

    Traverse its log records backward

```

```

FOR each update operation DO
    Undo the operation
    Write compensation log record
END FOR

```

```

Write END record

```

```

END FOR

```

```

END

```

ARIES phases:

Phase	Purpose
Analysis	Determines transaction and dirty-page status
Redo	Repeats necessary operations
Undo	Reverses incomplete transactions

Conclusion: ARIES restores the database to a consistent state after a system crash while preserving committed transactions.

8. Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

Answer:

Shadow paging maintains two page tables:

Shadow Page Table – points to the stable database pages.

Current Page Table – contains changes made by the transaction.

The shadow page table is not modified during normal transaction execution.

Pseudocode:

```

ALGORITHM Shadow_Paging_Recovery()

```

```

BEGIN

```

```

    // Initialization

```

```

    ShadowPageTable ← CurrentPageTable

```

WHILE Transaction is running DO

IF Transaction modifies Page P THEN

 Allocate New Page P'

 Copy contents of P to P'

 Apply modification to P'

 Update CurrentPageTable[P] $\leftarrow$ P'

END IF

END WHILE

IF Transaction COMMITs THEN

 Write CurrentPageTable to stable storage

 ShadowPageTable $\leftarrow$ CurrentPageTable

 Commit Transaction

ELSE IF System FAILURE occurs THEN

 CurrentPageTable $\leftarrow$ ShadowPageTable

 Discard modified pages

 Restart Transaction

END IF

END

Working:

Before Transaction

Shadow Page Table



Original Pages

During Transaction

Current Page Table



New Modified Pages

Failure



Discard Current Table



Restore Shadow Page Table

Conclusion: Shadow paging provides simple recovery because the original shadow pages remain unchanged until the transaction commits.

9. Write pseudocode for Immediate Update recovery using a UNDO/REDO log processing after a system crash.

Answer:

In immediate update, database changes can be written to disk before the transaction commits. Therefore, recovery must use the log to perform both UNDO and REDO operations.

Pseudocode:

ALGORITHM Immediate_Update_Recovery(Log)

BEGIN

 Read Log from the beginning or last checkpoint

 Committed ← Empty

 Active ← Empty

```

// Identify transaction states
FOR each Log_Record DO

    IF Log_Record = START(T) THEN
        Add T to Active

    ELSE IF Log_Record = COMMIT(T) THEN
        Move T from Active to Committed

    END IF

END FOR

// REDO committed transactions
FOR each Log_Record in forward order DO

    IF Record belongs to a Committed Transaction THEN

        IF Record is UPDATE(X, OldValue, NewValue) THEN
            Write NewValue to X
        END IF

    END IF

END FOR

// UNDO incomplete transactions
FOR each Log_Record in reverse order DO

    IF Record belongs to an Active Transaction THEN

        IF Record is UPDATE(X, OldValue, NewValue) THEN
            Write OldValue to X
        END IF

    END IF

END IF

```


END FOR

Display "Database Recovery Completed"

END

Example:

Log:

<T1 START>

<T1, A, 100, 200>

<T1 COMMIT>

<T2 START>

<T2, B, 300, 400>

CRASH

After recovery:

T1 → REDO → A = 200

T2 → UNDO → B = 300

Conclusion: REDO restores committed changes, while UNDO removes changes made by incomplete transactions.

10. Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

Answer:

A database connectivity manager maintains reusable database connections and handles transactions safely.

Pseudocode:

ALGORITHM Database_Connectivity_Manager

BEGIN

Initialize ConnectionPool

FUNCTION GetConnection()

IF Available connection exists THEN

Return connection

ELSE

Create new connection

Return connection

END IF

END FUNCTION

FUNCTION ExecuteQuery(Query)

Connection ← GetConnection()

TRY

Begin Transaction

Result ← Execute Query using Connection

Commit Transaction

Return Result

CATCH Exception

Rollback Transaction

Log Exception

Display "Query Failed"

FINALLY

Return Connection to ConnectionPool

END TRY

END FUNCTION

FUNCTION ClosePool()

FOR each Connection in ConnectionPool DO
 Close Connection
END FOR

END FUNCTION

END

Working:

Application



Get Connection



Connection Pool



Execute Query



Success? — Yes —→ COMMIT



No



ROLLBACK



Return Connection to Pool

Advantages:

- Reduces connection creation overhead.
- Reuses existing connections.
- Improves performance.
- Provides transaction rollback during exceptions.
- Prevents connection leaks.

Conclusion: A database connectivity manager provides efficient and reliable database access through connection pooling, query execution, and exception handling.